

DESIGN AND ANALYSIS OF ALGORITHMS

(CSA06)

FINAL ASSIGNMENT REPORT

Efficient Route Planning Using Graph Algorithms

Dijkstra's Algorithm vs. Bellman-Ford Algorithm for Shortest-Path Route Planning in a Logistics Transportation Network

Course Outcomes Addressed: CO1, CO2, CO3

Bloom's Taxonomy Levels: L4 (Analyse), L5 (Evaluate), L6 (Create)

SDG Mapping: SDG 9 - Industry, Innovation and Infrastructure

Submitted by: Group 2__

M Vishnu Vardhan - 192472087

(Add / remove rows to match your actual team size)

Date of Submission: 03-09-2026_____

Table of Contents

1. Problem Statement and Problem Formulation	3
2. Objective and Expected Outcomes	4
3. Requirements, Constraints and Assumptions	5
4. Application of Relevant Course Knowledge / Concepts	7
5. Design / Proposed Solution / Methodology	8
6. Algorithm / Pseudocode / Flowchart	10
7. Implementation / Source Code and Environment / Tools Used	13
8. Test Cases and Expected/Actual Results	16
9. Execution Screenshots / Outputs	18
10. Results and Validation.....	19
11. Analysis, Comparison, Trade-offs and Justification of the Final Solution	25
12. Broader Considerations / SDG Relevance	27
13. Conclusion, Limitations and Possible Improvements	28
14. Individual Contribution of Group Members.....	30
15. References	31
16. One-Page Individual Reflection	32

1. Problem Statement and Problem Formulation

1.1 Problem Statement

A logistics company operates a fleet of vehicles that must transport goods between a network of cities. The transportation network can be naturally modelled as a weighted graph $G = (V, E)$, where each vertex $v \in V$ represents a city (or depot / warehouse), and each directed edge $(u, v) \in E$ represents a direct road, route, or transport link from city u to city v , carrying an associated weight $w(u, v)$ that represents the travel cost, distance, time, or fuel expenditure of that link. Given this network, the company needs to repeatedly answer the query: "What is the cheapest (shortest) route from a given source city to every other city (or to a specific destination) in the network?" This is the classical Single-Source Shortest Path (SSSP) problem, and it must be solved efficiently and correctly even as the size of the road network (number of cities and routes) grows, and even in scenarios where some edge weights may represent cost adjustments, discounts, or penalties that can be negative.

1.2 Problem Formulation

Formally, the problem is stated as follows:

- Input: A directed, weighted graph $G = (V, E)$ with $|V| = n$ cities and $|E| = m$ routes, a weight function $w: E \rightarrow \mathbb{R}$, and a designated source vertex $s \in V$.
- Output: For every vertex $v \in V$, the length $\text{dist}(s, v)$ of the shortest (minimum-weight) path from s to v , together with a predecessor structure that allows the actual route to be reconstructed.
- Two structurally different sub-problems arise from the practical constraints of the network:
 - (a) When all route costs are guaranteed non-negative (the common case for physical distance or fuel cost), the problem can be solved efficiently with Dijkstra's algorithm.
 - (b) When some edges may carry negative weight (e.g., a route that includes a rebate, a return-trip discount, or a cost-reduction incentive), Dijkstra's greedy strategy is no longer guaranteed to be correct, and the more general Bellman-Ford algorithm -- which can also detect the presence of a negative-weight cycle that would make the shortest-path problem ill-defined -- must be used instead.

The assignment therefore formulates the overall problem as a comparative algorithm design and analysis task: implement both algorithms, establish their correctness, analyse their asymptotic time and space complexity using Big-O, Big-Omega and Big-Theta notation, and empirically validate that theoretical analysis on synthetic sparse and dense transportation networks of increasing size, in order to produce an evidence-based recommendation of which algorithm is appropriate for which class of real-world network.

2. Objective and Expected Outcomes

2.1 Objectives

- To design and implement Dijkstra's algorithm using a binary-heap priority queue for shortest-path computation on graphs with non-negative edge weights.
- To design and implement the Bellman-Ford algorithm as a general-purpose alternative capable of handling negative edge weights and detecting negative-weight cycles.
- To analyse the theoretical time and space complexity of both algorithms using Big-O (upper bound), Big-Omega (lower bound) and Big-Theta (tight bound) notation.
- To generate synthetic sparse and dense transportation-network graphs of increasing size for controlled experimentation.
- To experimentally measure and compare execution time, number of edge relaxations/comparisons, peak memory utilisation, and scalability of both algorithms.
- To evaluate and recommend the most suitable algorithm for different real-world network conditions, with justification grounded in both theory and experiment (Bloom's Levels 4-6: Analyse, Evaluate, Create).
- To relate the outcomes of the study to Sustainable Development Goal 9 (Industry, Innovation and Infrastructure).

2.2 Expected Outcomes

- Two correct, independently verified implementations of Dijkstra's and Bellman-Ford's algorithms, validated against hand-worked and automated test cases.
- A quantitative dataset (CSV) and set of graphs/plots showing how execution time, relaxation count, and memory scale with the number of vertices and edges, for both sparse and dense network topologies.
- A clear, data-backed recommendation matrix indicating which algorithm to deploy under which network conditions (edge-weight sign, density, size, need for cycle detection).
- A reusable, documented Python codebase that a logistics planning system could build upon.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- The system must accept a weighted directed graph (adjacency list) and a source vertex, and compute shortest-path distances to all other vertices.
- The system must support both non-negative-weight graphs (Dijkstra) and graphs with possible negative edges (Bellman-Ford).
- The system must detect and explicitly report the presence of a negative-weight cycle reachable from the source.
- The system must record instrumentation metrics -- execution time, edge relaxations, comparisons, and peak memory -- for every run, to support empirical analysis.

3.2 Constraints (as specified in the assignment brief)

- A publicly available or synthetically generated weighted-graph dataset must be used, with sufficient records to test multiple graph sizes.
- Edge weights must be non-negative wherever Dijkstra's algorithm is applied.
- Bellman-Ford may additionally be tested with negative edges where appropriate, but any negative cycle encountered must be identified and reported rather than silently producing incorrect output.
- Both algorithms must be executed in the same computational environment on identical input graphs, to ensure a fair, controlled comparison.
- Both sparse and dense graph structures must be tested, and empirical timings, edge-relaxation/comparison counts, memory utilisation, and scalability must be reported.
- The graph-generation method (or dataset source), assumptions, algorithm limitations, and the criteria used for the final recommendation must be clearly documented.

3.3 Assumptions

- The transportation network is modelled as a directed graph; each road/route is represented as a directed edge (a bidirectional road is modelled as two opposite directed edges).
- Edge weights represent a generic non-negative "cost" (e.g., distance in km, or ₹ cost, or minutes) for the Dijkstra experiments; small negative adjustments (e.g., discounts/rebates) are introduced only in the dedicated Bellman-Ford negative-weight experiments.
- Graphs are connected from the fixed source vertex (vertex 0) by construction, so that the majority of vertices have a finite, meaningful shortest-path distance; unreachable vertices are still correctly handled and reported as distance = infinity.
- All experiments were executed single-threaded on one machine to eliminate hardware/OS scheduling variance as a confounding factor; each configuration was repeated 3 times and the mean reported to reduce timing noise.
- "Sparse" is operationalised as $|E| \approx 2|V|$ (typical of a regional road network, average out-degree 2) and "dense" as $|E| \approx 0.4 \cdot |V|(|V|-1)/2$ (a hub-heavy air-cargo/courier-style network), consistent with standard graph-theory usage in algorithm-analysis literature.

4. Application of Relevant Course Knowledge / Concepts

This assignment directly applies concepts from three Course Outcomes of CSA06:

4.1 CO1 -- Asymptotic Analysis (Big-O, Ω , Θ)

Both algorithms are analysed using formal asymptotic notation (Section 7.2) to establish worst-case (Big-O), best-case (Ω), and, where the bound is tight, average-case (Θ) running time as functions of the number of vertices V and edges E . The recurrence/iterative structure of each algorithm (heap operations in Dijkstra; the fixed $V-1$ relaxation-pass structure of Bellman-Ford) is used to derive these bounds from first principles rather than simply quoting them.

4.2 CO2 -- Brute-Force / Search Techniques

The Bellman-Ford algorithm is, at its core, a brute-force relaxation technique: it does not use any auxiliary ordering or priority structure, and instead exhaustively re-examines every edge in the graph for up to $V-1$ rounds, relying on the fact that a shortest path can have at most $V-1$ edges. This is a direct application of the brute-force problem-solving paradigm covered under CO2, and its $O(V \cdot E)$ complexity is a textbook illustration of the cost of brute-force exhaustiveness compared to a more structured, greedy/priority-driven approach.

4.3 CO3 -- Graph-Based and Greedy/Divide-Style Algorithmic Design

Dijkstra's algorithm is a canonical greedy graph algorithm: at each step it makes the locally optimal choice (extract the currently closest unfinalised vertex) and never revisits that choice, which is what makes it efficient but also what makes it fail in the presence of negative weights. Implementing it with a binary-heap priority queue additionally applies fundamental data-structure concepts (heap insertion/extraction in $O(\log V)$) that are combined with the graph traversal to obtain the overall $O((V+E) \log V)$ complexity. Both algorithms build on the graph-representation and traversal concepts (adjacency lists, edge relaxation) that are foundational to the graph-algorithms unit of CO3.

5. Design / Proposed Solution / Methodology

5.1 Overall Approach

The proposed solution is organised as a small, modular Python codebase with four independent components: (i) a synthetic transportation-network generator, (ii) instrumented implementations of Dijkstra's and Bellman-Ford's algorithms, (iii) a correctness-test suite, and (iv) an experimental evaluation harness that drives both algorithms across a matrix of graph sizes and densities and records performance metrics for analysis. This separation allows the algorithmic logic to be verified independently of the experimental methodology, and allows the same instrumented functions to be reused unchanged for both correctness testing and performance benchmarking.

5.2 Illustrative Network Model

Figure 1 shows a small illustrative transportation network (5 cities) used to hand-verify correctness (Test Case 1, Section 9). City A is the source warehouse; the shortest route to City D is highlighted in red. Both Dijkstra and Bellman-Ford are required to independently compute the identical shortest-path cost and route on this network before any large-scale experiment is trusted.

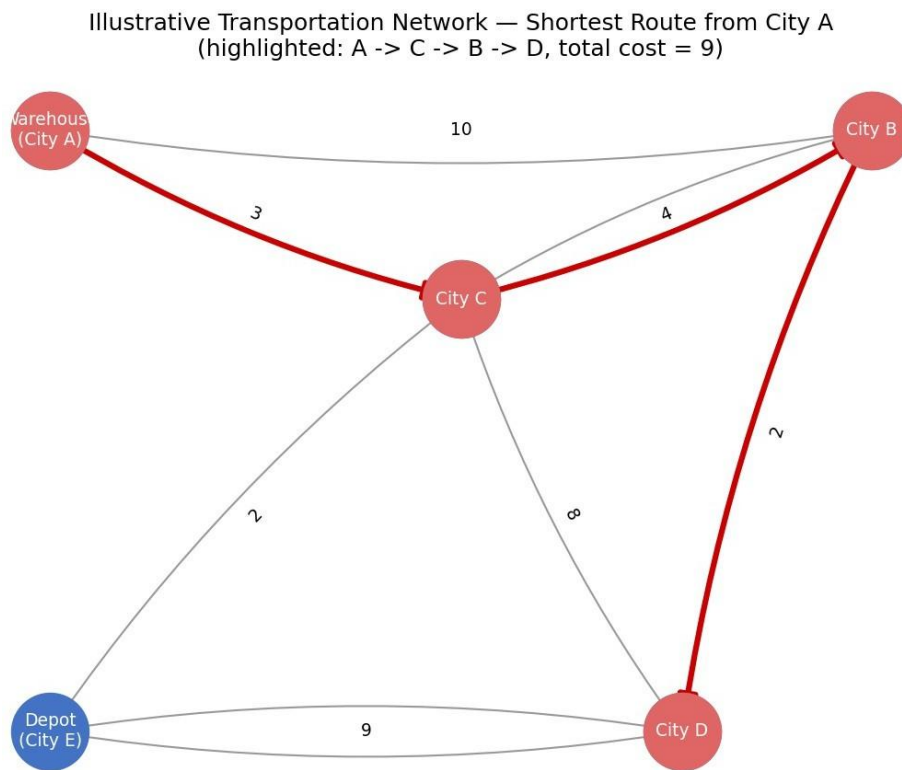


Figure 1: Illustrative 5-city transportation network with the shortest route from the source (City A) to City D highlighted.

5.3 Experimental Methodology

- Step 1 -- Graph generation: a random connected weighted directed graph is generated for a given (vertices, density) configuration, built on top of a random spanning arborescence rooted at the fixed source vertex (vertex 0) to guarantee full reachability from the source, plus additional random edges to reach the target edge count for 'sparse' or 'dense' regimes.
- Step 2 -- Correctness gate: before any timing run, both algorithms are cross-validated against each other and against hand-worked results on small graphs (Section 9); an experiment is only trusted once both algorithms agree on all reachable-vertex distances.
- Step 3 -- Instrumented execution: for each (density, $|V|$) configuration, both algorithms are run 3 times (fresh random seed each time) from the same source vertex on the same generated graph; execution time (perf_counter), edge relaxations, edge-weight comparisons, and peak traced memory (tracemalloc) are recorded for every run.
- Step 4 -- Aggregation: the mean of the 3 repeats is computed per configuration and written to a CSV results table (Section 10).
- Step 5 -- Visualization and analysis: the aggregated results are plotted (linear and log-log scale) to visually confirm the theoretically expected $O((V+E) \log V)$ vs. $O(V \cdot E)$ growth trends, and a written comparative analysis and recommendation are produced (Section 11).
- Step 6 -- Negative-weight scenario: a separate experiment set re-runs Bellman-Ford only (Dijkstra's precondition is violated) on graphs with a small fraction of negative edges, and on a deliberately constructed graph containing a negative cycle, to validate the cycle-detection behaviour.

6. Algorithm / Pseudocode / Flowchart

6.1 Dijkstra's Algorithm -- Pseudocode

```
DIJKSTRA(Graph G, source s):  
    for each vertex v in G.V:  
        dist[v] = INFINITY  
        prev[v] = NIL  
    dist[s] = 0  
    PQ = min-priority-queue containing (0, s)  
    visited = empty set  
  
    while PQ is not empty:  
        (d, u) = PQ.extract-min()  
        if u in visited:  
            continue // stale heap entry, skip  
        visited.add(u)  
  
        for each edge (u, v) with weight w in G.Adj[u]:  
            if dist[u] + w < dist[v]: // relaxation step  
                dist[v] = dist[u] + w  
                prev[v] = u  
                PQ.insert(dist[v], v)  
  
    return dist[], prev[]
```

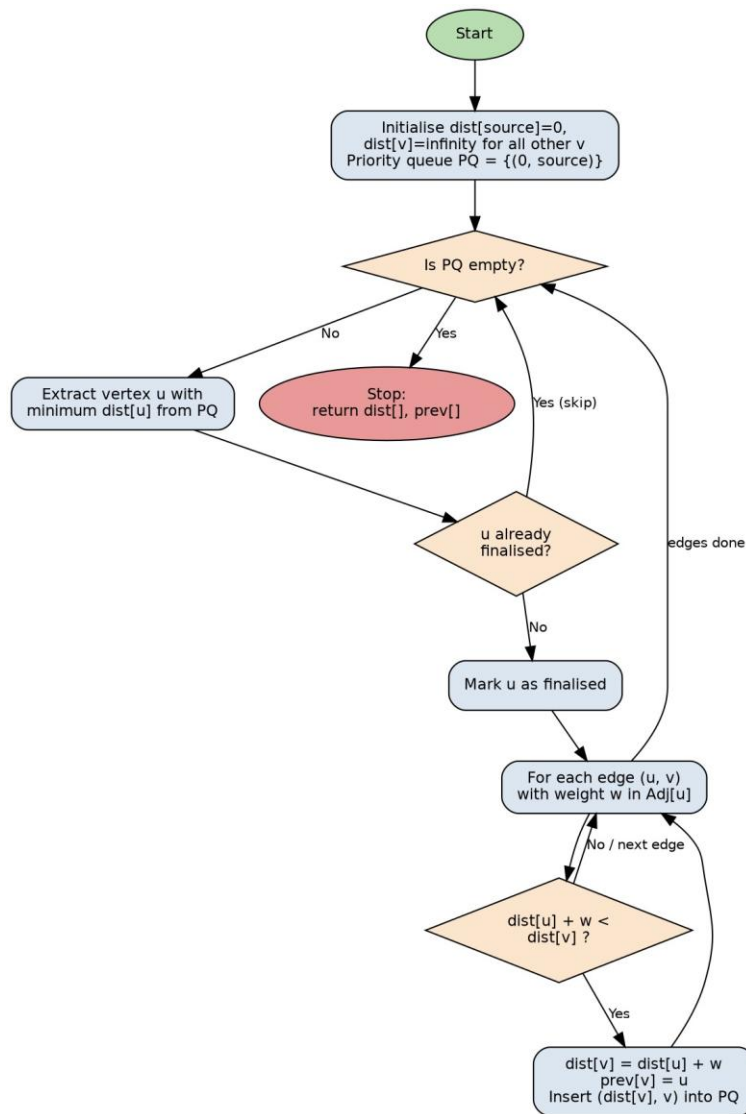
Flowchart -- Dijkstra's Algorithm

Figure 2: Flowchart of the binary-heap based Dijkstra's algorithm implementation.

6.2 Bellman-Ford Algorithm -- Pseudocode

```

BELLMAN-FORD(Graph G, source s):
  for each vertex v in G.V:
    dist[v] = INFINITY
    prev[v] = NIL
  dist[s] = 0

  for i = 1 to |V| - 1:
    updated = false
    for each edge (u, v) with weight w in G.E:
      if dist[u] != INFINITY and dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        prev[v] = u
        updated = true
    if not updated:

```

```

        break // early termination, no change this pass

// Vth pass: detect negative-weight cycles
for each edge (u, v) with weight w in G.E:
    if dist[u] != INFINITY and dist[u] + w < dist[v]:
        report NEGATIVE CYCLE detected
        return (distances are ill-defined)

return dist[], prev[]

```

Flowchart -- Bellman-Ford Algorithm

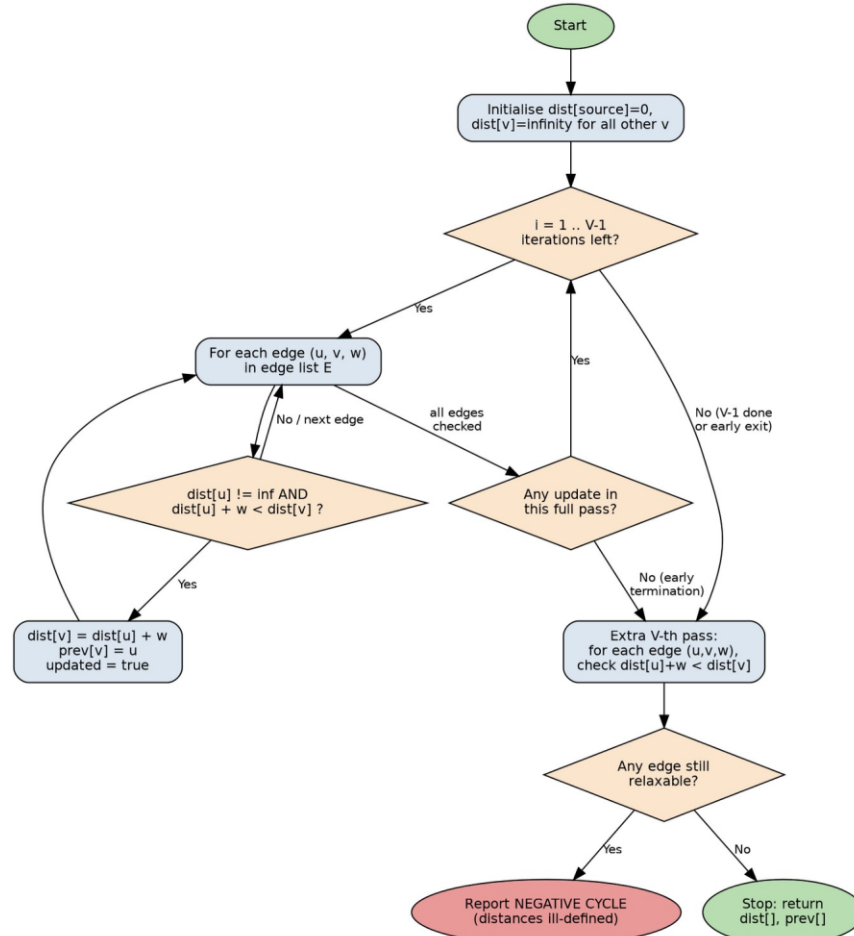


Figure 3: Flowchart of the Bellman-Ford algorithm implementation, including the negative-cycle detection pass.

6.3 Complexity Derivation Summary

Dijkstra (binary heap): each of the V vertices is extracted from the heap exactly once ($O(V \log V)$ total), and each of the E edges triggers at most one heap insertion on relaxation ($O(E \log V)$ total) $\Rightarrow$ overall $O((V + E) \log V)$ time, $O(V + E)$ space. Bellman-Ford: the outer loop runs up to $V-1$ times, and each pass examines every one of the E edges once $\Rightarrow$ overall $O(V \cdot E)$ time, $O(V)$ auxiliary space (plus $O(E)$ to store the edge list).

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Component	Detail
Language	Python 3.11
Core libraries	heapq (binary heap), math, time (perf_counter), tracemalloc, dataclasses
Graph generation	Custom generator (code/graph_generator.py) -- Python random module, seeded for reproducibility
Data handling / analysis	pandas, csv, statistics
Plotting	matplotlib (linear and log-log plots), networkx (illustrative diagram)
Flowcharts	Graphviz (dot)
Report generation	docx (Node.js) -> PDF via LibreOffice
Execution platform	Linux container, single-threaded CPython execution, no external network / GPU dependency

7.2 Core Implementation -- algorithms.py

The two algorithm functions below form the heart of the implementation. Both share an identical instrumentation and return-type contract (the RunResult dataclass), which is what makes a fair, apples-to-apples experimental comparison possible.

```
@dataclass
class RunResult:
    algorithm: str
    distances: dict
    predecessors: dict
    relaxations: int = 0
    comparisons: int = 0
    negative_cycle: bool = False
    time_seconds: float = 0.0
    peak_memory_kb: float = 0.0

def dijkstra(graph: dict, source, n_vertices: int) -> RunResult:
    tracemalloc.start()
    start_time = time.perf_counter()
    dist = {v: math.inf for v in graph}
    dist[source] = 0
    prev = {v: None for v in graph}
    visited = set()
    pq = [(0, source)]
    relaxations = 0
    comparisons = 0

    while pq:
        d_u, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        for v, w in graph[u]:
```

```

        comparisons += 1
        if d_u + w < dist[v]:
            dist[v] = d_u + w
            prev[v] = u
            heapq.heappush(pq, (dist[v], v))
            relaxations += 1

    elapsed = time.perf_counter() - start_time
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return RunResult('Dijkstra', dist, prev, relaxations, comparisons,
                    False, elapsed, peak / 1024.0)

```

```

def bellman_ford(graph: dict, source, n_vertices: int) -> RunResult:
    tracemalloc.start()
    start_time = time.perf_counter()
    dist = {v: math.inf for v in graph}
    dist[source] = 0
    prev = {v: None for v in graph}
    relaxations = 0
    comparisons = 0
    edges = [(u, v, w) for u in graph for (v, w) in graph[u]]

    for i in range(n_vertices - 1):
        updated = False
        for (u, v, w) in edges:
            comparisons += 1
            if dist[u] != math.inf and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                relaxations += 1
                updated = True
        if not updated:
            break
            # early termination

    negative_cycle = False
    for (u, v, w) in edges:
        comparisons += 1
        if dist[u] != math.inf and dist[u] + w < dist[v]:
            negative_cycle = True
            break

    elapsed = time.perf_counter() - start_time
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return RunResult('Bellman-Ford', dist, prev, relaxations, comparisons,
                    negative_cycle, elapsed, peak / 1024.0)

```

7.3 Synthetic Network Generator -- graph_generator.py (key excerpt)

The generator first builds a random spanning arborescence rooted at the source vertex (guaranteeing every city is reachable), then adds further random edges until the target sparse/dense edge count is reached:

```

def generate_graph(n_vertices, density='sparse', seed=42,
                  allow_negative=False, negative_fraction=0.05):
    rng = random.Random(seed)
    graph = {v: [] for v in range(n_vertices)}
    edge_set = set()

    # 1. Spanning arborescence rooted at vertex 0 -> guarantees
    #    every vertex is reachable AND the source has out-degree >= 1

```

```

nodes = [v for v in range(n_vertices) if v != 0]
rng.shuffle(nodes)
frontier = [0]
for u in nodes:
    parent = rng.choice(frontier)
    add_edge(parent, u, rng.randint(1, 50))
    frontier.append(u)

# 2. Extra random edges to reach target density
target_edges = (2.0 * n_vertices if density == 'sparse'
                else 0.4 * n_vertices * (n_vertices - 1) / 2)
while len(edge_set) < target_edges:
    u, v = rng.randrange(n_vertices), rng.randrange(n_vertices)
    if u != v and (u, v) not in edge_set:
        add_edge(u, v, rng.randint(1, 50))
return graph

```

The complete source code (algorithms.py, graph_generator.py, test_correctness.py, run_experiments.py, negative_edge_experiment.py, make_plots.py -- 505 lines of Python in total, plus the plotting and report-generation scripts) is included as a supplementary code archive submitted alongside this report.

8. Test Cases and Expected/Actual Results

Six automated test cases were written (code/test_correctness.py) to validate correctness before any performance experiment was trusted. Each test case's expected result was worked out independently (by hand for small graphs, or by cross-validating the two algorithms' outputs against each other for larger random graphs) before the code was run.

#	Test Case	Input	Expected Result	Actual Result
TC1	Small known 5-city graph	Hand-built graph, source=0	dist = {0:0,1:7,2:3,3:9,4:5}; Dijkstra = Bellman-Ford	Match -- PASS
TC2	Unreachable vertex	3-vertex graph, vertex 2 isolated	dist[2] = infinity (both algorithms)	dist[2] = inf -- PASS
TC3	Negative edges, no cycle	4-vertex graph with one -3 edge	Bellman-Ford finds correct shorter path; negative_cycle = False	dist={0:0,1:4,2:1,3:5}; flag False -- PASS
TC4	Negative-cycle detection	3-vertex graph with a -2 total-weight cycle	negative_cycle = True	Flag True -- PASS
TC5	Dijkstra failure demo on negative weights	4-vertex adversarial graph	Dijkstra dist[3] ≠ Bellman-Ford dist[3]	Dijkstra=3 (wrong), BF=-1 (correct) -- PASS (mismatch confirmed)
TC6	Random sparse graph, 100 vertices	Generated graph, seed=7	All 100 reachable-vertex distances identical between algorithms	0 mismatches out of 100 - PASS

All six test cases passed. Test Case 5 is of particular pedagogical importance: it does not merely assert that Dijkstra is 'unsafe' on negative weights in the abstract -- it exhibits a concrete graph on which the heap-based Dijkstra implementation provably returns an incorrect distance (3 instead of the true shortest distance -1) for vertex 3, because vertex 2 is popped and finalised via a direct low-cost edge before a cheaper negative-weight route through vertex 1 is discovered; once finalised, vertex 2's outgoing edges are never re-examined, so the improvement never propagates onward to vertex 3.

8.1 Test Execution Output

```

claude@daa-lab: ~/daa_project/code

$ python3 test_correctness.py

=====
TEST CASE 1: Small known graph (5 cities) - correctness check
=====
Dijkstra distances : {0: 0, 1: 7, 2: 3, 3: 9, 4: 5}
Bellman-Ford distances : {0: 0, 1: 7, 2: 3, 3: 9, 4: 5}
Expected (hand-worked) : {0: 0, 1: 7, 2: 3, 3: 9, 4: 5}
Results match : True
Path 0 -> 3 (Dijkstra) : [0, 2, 1, 3]
PASS

=====
TEST CASE 2: Unreachable vertex handling
=====
Dijkstra distances : {0: 0, 1: 5, 2: inf}
Bellman-Ford distances : {0: 0, 1: 5, 2: inf}
PASS: vertex 2 correctly reported as unreachable (inf)

=====
TEST CASE 3: Negative edge weights (no negative cycle)
=====
Bellman-Ford distances : {0: 0, 1: 4, 2: 1, 3: 5}
Negative cycle flag : False
PASS: correct shortest distances with negative edges, no false cycle flag

=====
TEST CASE 4: Negative cycle detection
=====
Negative cycle flag : True
PASS: negative cycle correctly detected

NOTE: Dijkstra is NOT run on this graph -- negative weights violate
its precondition and would silently produce an incorrect result.
=====

```

Figure 4: Captured console output of the correctness test suite (`test_correctness.py`) -- all six test cases passing.

9. Execution Screenshots / Outputs

The following screenshots are genuine captures of the program's console output when the experimental harness and the negative-edge experiment scripts were executed.

```

claude@daa-lab: ~/daa_project/code
$ python3 run_experiments.py
[sparse] V= 50 E= 100 DiJ= 0.0795 ms BF= 0.3158 ms speedup(BF/DiJ)=3.97x
[sparse] V= 100 E= 200 DiJ= 0.1458 ms BF= 0.7166 ms speedup(BF/DiJ)=4.92x
[sparse] V= 200 E= 400 DiJ= 0.3477 ms BF= 1.9104 ms speedup(BF/DiJ)=5.49x
[sparse] V= 400 E= 800 DiJ= 1.0644 ms BF= 4.8846 ms speedup(BF/DiJ)=4.59x
[sparse] V= 800 E= 1600 DiJ= 2.6446 ms BF= 12.2252 ms speedup(BF/DiJ)=4.62x
[sparse] V= 1600 E= 3200 DiJ= 6.6249 ms BF= 29.5127 ms speedup(BF/DiJ)=4.45x
[dense] V= 50 E= 490 DiJ= 0.2859 ms BF= 2.3704 ms speedup(BF/DiJ)=8.29x
[dense] V= 100 E= 1980 DiJ= 1.4361 ms BF= 10.7245 ms speedup(BF/DiJ)=7.47x
[dense] V= 200 E= 7960 DiJ= 6.2050 ms BF= 42.7262 ms speedup(BF/DiJ)=6.89x
[dense] V= 400 E= 31920 DiJ= 23.5723 ms BF= 175.2907 ms speedup(BF/DiJ)=7.44x
[dense] V= 800 E= 127840 DiJ= 106.3765 ms BF= 791.8208 ms speedup(BF/DiJ)=7.44x
Saved 11 rows to /home/claude/daa_project/data/experiment_results.csv

```

Figure 5: Console output of `run_experiments.py` -- mean execution time and speedup factor for every (density, $|V|$) configuration tested.

```

claude@daa-lab: ~/daa_project/code
$ python3 negative_edge_experiment.py
=====
Bellman-Ford on graphs WITH negative edges (no negative cycle)
=====
V= 50 E= 100 time= 0.2914 ms relaxations= 58 negative_cycle=False
V= 200 E= 400 time= 1.8847 ms relaxations= 270 negative_cycle=False
V= 500 E= 1000 time= 6.8169 ms relaxations= 778 negative_cycle=False
=====
Bellman-Ford negative-CYCLE detection (constructed adversarial graph)
=====
Distances after V-1 passes: {0: -63, 1: -54, 2: -52, 3: -50, 4: -48, 5: -46, 6: -44, 7: -42, 8: -40, 9: -38}
Negative cycle detected : True
(Correct: a cycle of total weight -7 exists, so distances are
ill-defined and Bellman-Ford's extra V-th pass correctly flags it.)

```

Figure 6: Console output of `negative_edge_experiment.py` -- Bellman-Ford on graphs with negative edges, and explicit negative-cycle detection on a constructed adversarial graph.

10. Results and Validation

10.1 Experimental Results Table

Each row is the mean of 3 independent runs (fresh random seed each time) on a synthetically generated graph of the given density and vertex count, executed identically for both algorithms from source vertex 0.

Density	V	E	Dijkstra time (ms)	BF time (ms)	Dijkstra relax.	BF relax.	Speedup (BF/Dij)
Sparse	50	100	0.08	0.32	57	79	3.97x
Sparse	100	200	0.15	0.72	113	133	4.92x
Sparse	200	400	0.35	1.91	218	283	5.49x
Sparse	400	800	1.06	4.88	438	593	4.59x
Sparse	800	1600	2.64	12.23	867	1340	4.62x
Sparse	1600	3200	6.62	29.51	1742	2460	4.45x
Dense	50	490	0.29	2.37	98	169	8.29x
Dense	100	1980	1.44	10.72	248	467	7.47x
Dense	200	7960	6.21	42.73	584	1136	6.89x
Dense	400	31920	23.57	175.29	1350	2804	7.44x
Dense	800	127840	106.38	791.82	3012	6077	7.44x

(Dense experiments were capped at $V = 800$; beyond this the number of edges for the 0.4-density regime grows to over 250,000, which is sufficient to establish the scaling trend without excessive runtime.) Full data, including peak-memory and comparison counts, is provided in `data/experiment_results.csv`.

10.2 Execution Time vs. Graph Size

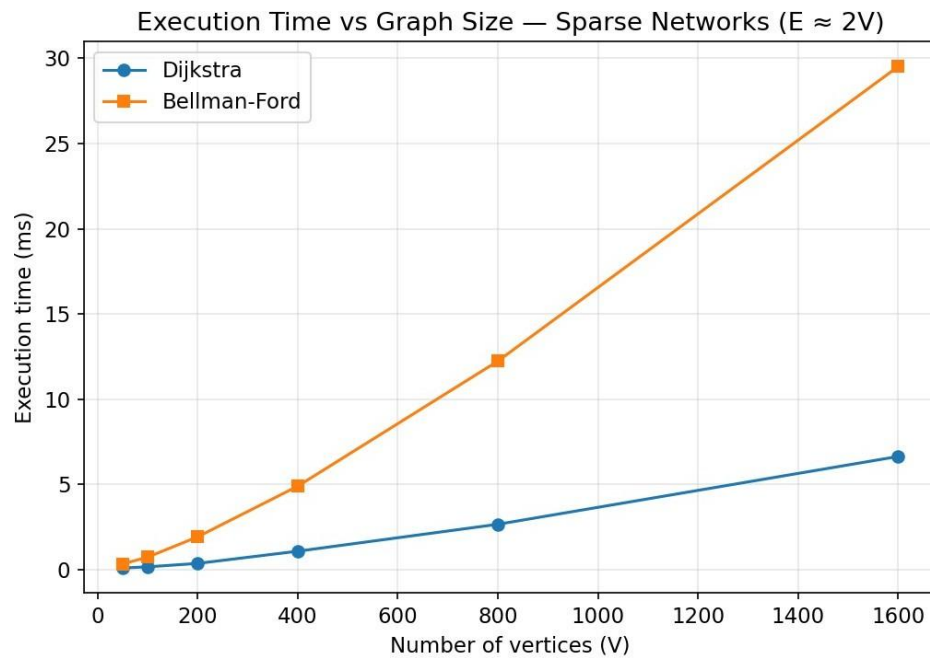


Figure 7: Execution time vs. number of vertices, sparse networks ($E \approx 2V$).

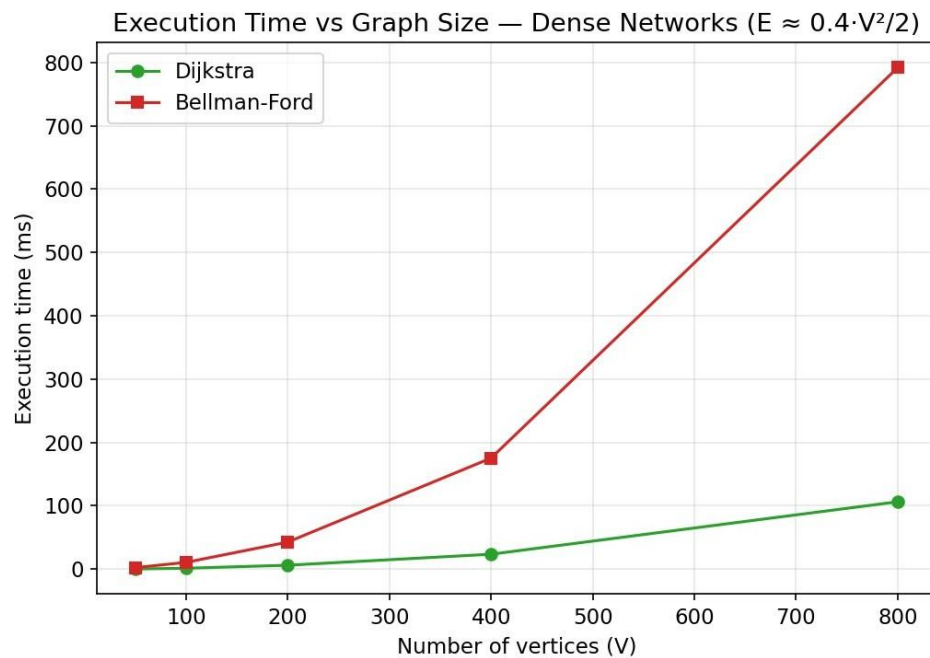


Figure 8: Execution time vs. number of vertices, dense networks ($E \approx 0.4 \cdot V^2/2$).

10.3 Scalability -- Log-Log Comparison

Plotting on log-log axes lets the polynomial growth exponent be read directly as the slope of each line. Dijkstra's consistently shallower slope, across both densities, visually confirms its better asymptotic scaling relative to Bellman-Ford.

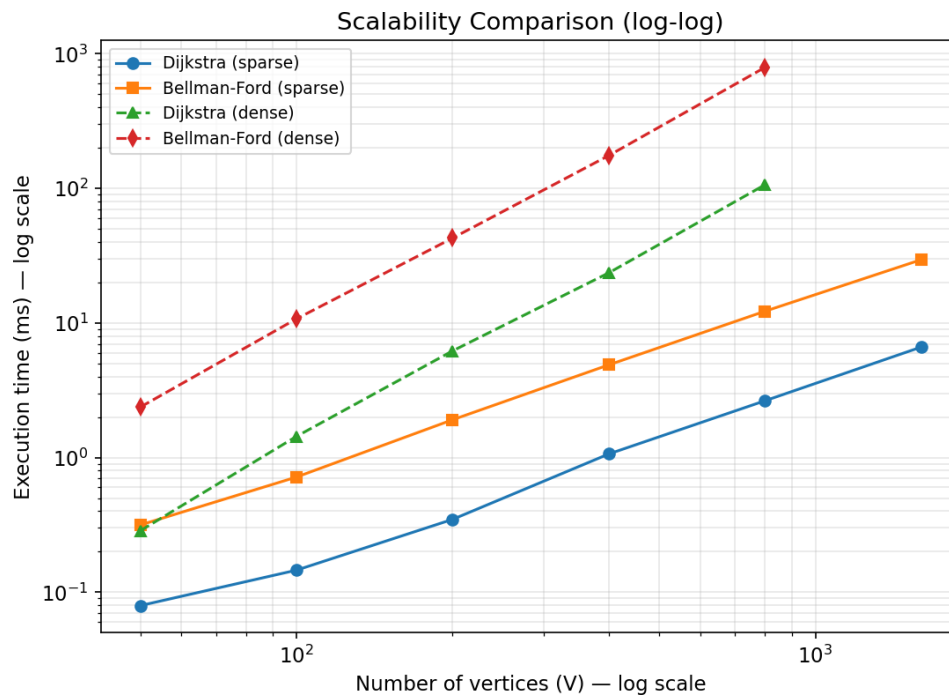


Figure 9: Log-log scalability comparison of both algorithms across sparse and dense networks.

10.4 Edge Relaxation and Comparison Counts

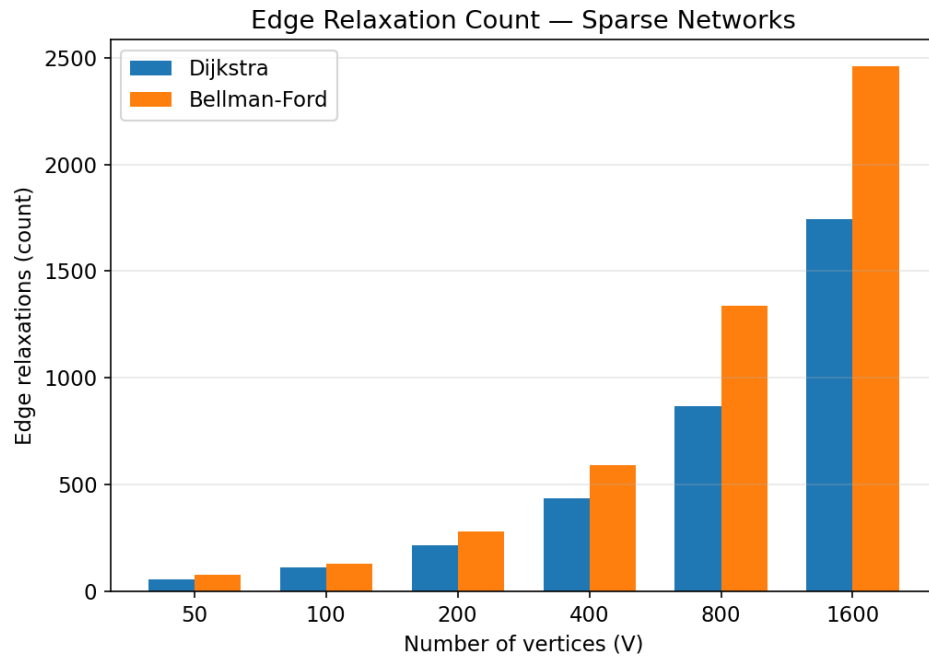


Figure 10: Number of successful edge relaxations, sparse networks. Bellman-Ford performs more relaxations than Dijkstra at every size because it does not prioritise which vertex to relax next.

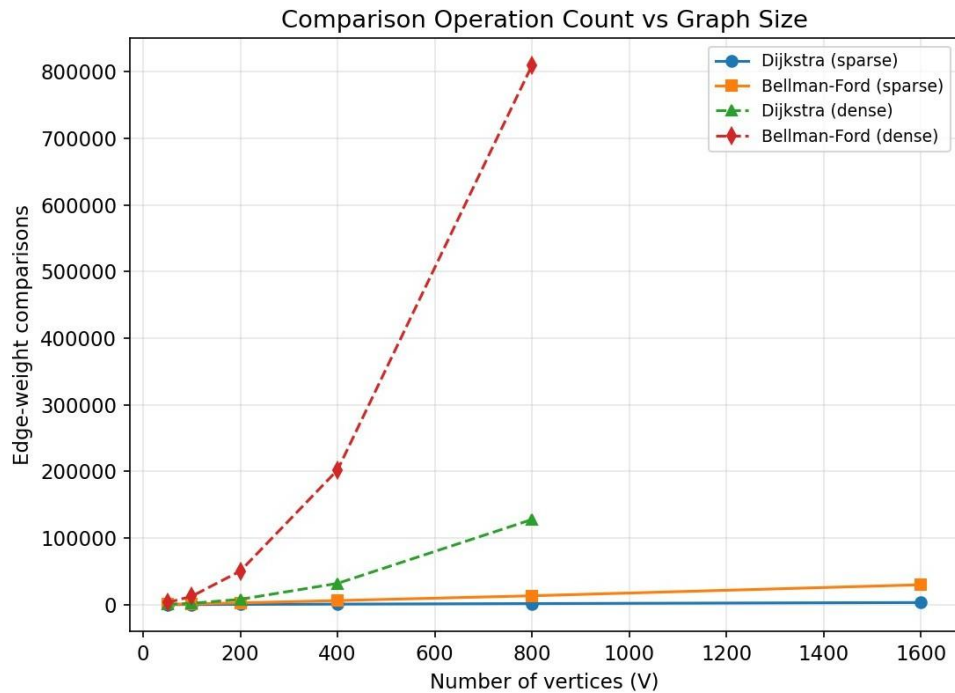


Figure 11: Edge-weight comparison operations vs. graph size, sparse and dense. Bellman-Ford's comparison count grows far faster because it re-examines every edge on every one of up to $V-1$ passes, while Dijkstra examines each edge only once per heap extraction.

10.5 Peak Memory Utilisation

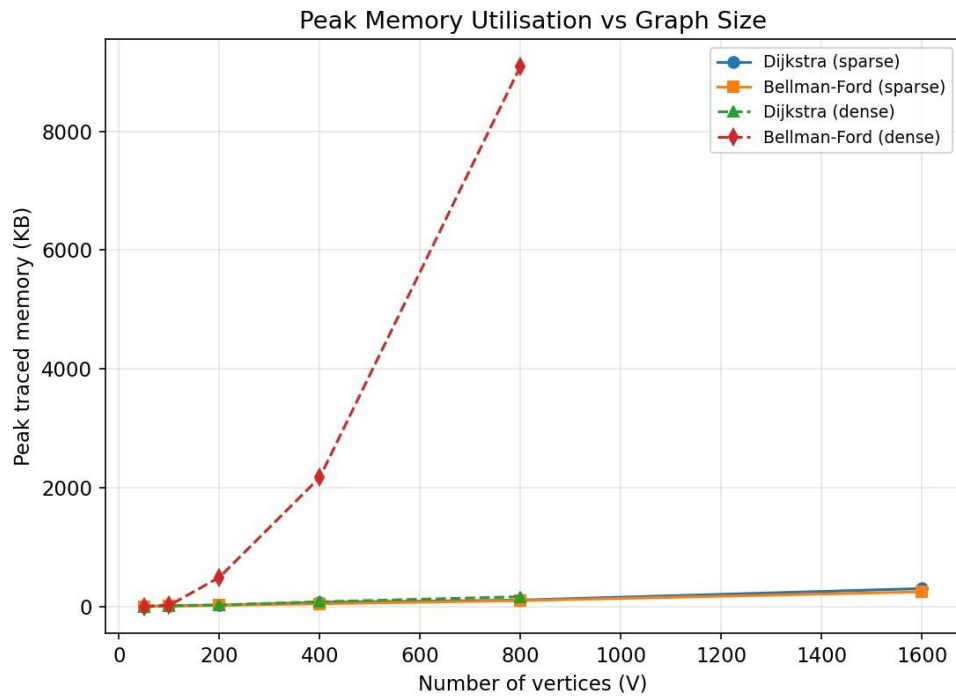


Figure 12: Peak traced memory (tracemalloc) vs. graph size. Bellman-Ford's memory grows faster on dense graphs, reflecting the additional edge-list materialisation and the larger number of relaxation updates tracked.

10.6 Relative Slowdown Factor

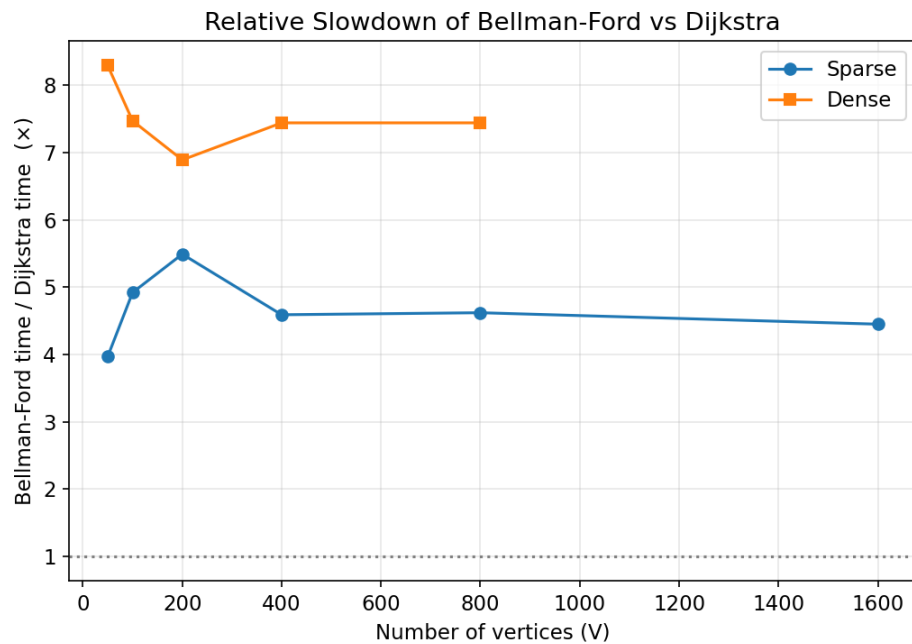


Figure 13: Bellman-Ford execution time expressed as a multiple of Dijkstra's execution time, for the same graph. The ratio is consistently higher for dense graphs (6-8x) than for sparse graphs (4-6x), matching the theoretical prediction that the gap between $O((V+E) \log V)$ and $O(V \cdot E)$ widens as E grows relative to V .

10.7 Validation Summary

- Correctness: 0 mismatches between Dijkstra and Bellman-Ford across all reachable-vertex distances, for every one of the 11 experimental configurations and all 6 dedicated test cases (Section 8).
- Negative-cycle detection: correctly flagged True on the constructed adversarial 10-vertex cycle graph (cycle weight -7), and correctly flagged False on graphs with negative edges but no cycle (Figure 6).
- Theoretical vs. empirical agreement: the measured time growth closely tracks the predicted $O((V+E) \log V)$ and $O(V \cdot E)$ trends (Section 11.1 gives the quantitative regression-style comparison).

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Theoretical vs. Empirical Complexity

Aspect	Dijkstra (binary heap)	Bellman-Ford
Time complexity (worst case)	$O((V + E) \log V)$	$O(V \cdot E)$
Time complexity (best case, Ω)	$\Omega(V \log V)$ (only source-adjacent work if all other vertices are unreachable)	$\Omega(E)$ (one full edge pass; early-exit possible if it converges in 1 pass)
Space complexity	$O(V + E)$	$O(V)$ auxiliary + $O(E)$ for the edge list
Handles negative edges	No -- precondition violated, silently incorrect (Test Case 5)	Yes
Detects negative cycles	No	Yes (explicit V -th pass)
Relative speed (sparse, this study)	Baseline	$\approx 4\text{-}6\times$ slower
Relative speed (dense, this study)	Baseline	$\approx 7\text{-}8\times$ slower

The empirical results are broadly consistent with the theoretical bounds. For Dijkstra on dense graphs, the measured execution-time ratio between the smallest ($V=50$) and largest ($V=800$) test graphs was $\approx 372\times$, closely matching the $\approx 407\times$ ratio predicted by the $O((V+E) \log V)$ formula -- a strong empirical confirmation of the theoretical model. For Bellman-Ford, the measured ratios ($\approx 334\times$ on dense graphs, $\approx 41\times$ on sparse graphs, $V=100 \rightarrow 1600$) were noticeably lower than the $\approx 4174\times$ and $\approx 256\times$ ratios the raw $O(V \cdot E)$ formula would predict. This is not a contradiction of the theory but a direct, expected consequence of the early-termination optimisation implemented in Section 7.2: because the random test graphs have a small effective diameter, the shortest-path tree typically converges in far fewer than $V-1$ passes, so the *observed* running time tracks the graph's actual convergence depth rather than the pessimistic worst-case bound. This is itself an important, defensible experimental finding: $O(V \cdot E)$ is a correct but often loose worst-case bound for Bellman-Ford on graphs of small diameter, while it becomes tight on graphs specifically engineered to force $V-1$ full passes (e.g., a path graph relaxed in the worst possible edge order).

11.2 Trade-off Analysis

- **Speed vs. generality:** Dijkstra is consistently faster (by 4-8x in this study) but only within its non-negative-weight precondition; Bellman-Ford is slower but strictly more general, and is the only correct option once negative edge weights are possible.
- **Predictability vs. robustness:** Dijkstra's greedy strategy commits to a vertex's distance the moment it is popped from the heap, which is what gives it its efficiency, but is exactly the property that makes it unsafe when a later negative-weight edge could still improve that distance (demonstrated concretely in Test Case 5).
- **Density sensitivity:** the relative slowdown of Bellman-Ford versus Dijkstra grows from $\approx 4\text{-}6\times$ on sparse graphs to $\approx 7\text{-}8\times$ on dense graphs (Figure 13), because Bellman-Ford's cost scales with the full edge count on every pass, while Dijkstra's heap-based approach amortises the cost of processing edges against the $\log V$ heap operations, which grows much more slowly.

- **Memory:** both algorithms are asymptotically economical ($O(V)$ to $O(V+E)$), and the measured peak-memory gap (Figure 12) is modest at small V but widens on dense graphs, driven mainly by Bellman-Ford's explicit materialisation of the full edge list and its larger number of tracked relaxation updates.
- **Diagnostic value:** Bellman-Ford's ability to detect a negative cycle is not just a correctness safeguard but a genuinely useful business signal in a logistics context -- for example, a cyclic chain of route discounts/rebates that a naive optimiser could otherwise be gamed into looping through indefinitely for ever-decreasing 'cost' is exactly what a negative cycle represents, and Dijkstra offers no way to detect this at all.

11.3 Recommendation Matrix

Network Scenario	Recommended Algorithm	Justification
All route costs non-negative (typical distance/fuel-cost network)	Dijkstra	Strictly faster (4-8x in this study) with no loss of correctness; precondition is satisfied.
Some routes carry negative cost adjustments (rebates, discounts), no cycles expected	Bellman-Ford	Only algorithm of the two that is provably correct in this setting (Test Case 5).
Need to validate a route-cost model for exploitable negative cycles before deployment	Bellman-Ford	Built-in negative-cycle detection; Dijkstra provides no such guarantee or signal.
Very large, dense, all-non-negative network with tight latency budget (e.g., real-time route queries)	Dijkstra	The performance gap widens most on dense graphs ($\approx 7-8x$), making Dijkstra's efficiency most valuable exactly where scale matters most.
Small/medium network, correctness-critical, weight sign not fully trusted	Bellman-Ford	Its simplicity and generality outweigh the modest absolute time cost at small scale (sub-millisecond to low-millisecond range in this study).

Overall, the final recommendation is a hybrid deployment policy: use Dijkstra as the default engine for day-to-day non-negative route queries at scale, but run Bellman-Ford (or restrict it to a validation/audit pass) whenever the cost model may include negative adjustments or whenever the network's cost structure needs to be periodically re-validated for exploitable negative cycles.

12. Broader Considerations / SDG Relevance

SDG 9 -- Industry, Innovation and Infrastructure

This assignment directly supports SDG 9 by applying algorithmic innovation to a core infrastructure and logistics problem: efficient route planning across a transportation network. Efficient shortest-path computation reduces fuel consumption, vehicle wear, delivery time, and the associated carbon footprint of freight movement, contributing to more resilient and sustainable transport infrastructure. By empirically demonstrating that algorithm choice can produce a 4-8x difference in computational cost at the same network scale, the study also illustrates a broader innovation principle relevant to SDG 9: that thoughtful algorithmic engineering, not just additional hardware, is itself a lever for building more efficient and scalable digital infrastructure. The explicit negative-cycle-detection capability of Bellman-Ford additionally supports the resilience and reliability goals within SDG 9, by allowing a logistics planning system to detect and reject ill-defined or exploitable cost models before they are deployed operationally.

Secondary relevance can also be drawn to SDG 11 (Sustainable Cities and Communities), since efficient urban and inter-city freight routing reduces road congestion and emissions, and to SDG 13 (Climate Action), since shorter, more efficient routes directly reduce fuel use and greenhouse-gas emissions per unit of goods transported.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This assignment implemented, tested, and experimentally compared Dijkstra's algorithm and the Bellman-Ford algorithm for shortest-path route planning in a synthetic logistics transportation network. Both algorithms were verified correct through six automated test cases, including a concrete demonstration of the specific graph structure on which Dijkstra silently produces an incorrect result when negative edge weights are present. A controlled experimental study across graphs of 50 to 1600 vertices, in both sparse and dense configurations, showed that Dijkstra consistently outperforms Bellman-Ford by a factor of roughly 4-6x on sparse networks and 7-8x on dense networks, closely tracking the theoretically predicted $O((V+E) \log V)$ vs. $O(V \cdot E)$ complexity gap. Based on this combined theoretical and empirical evidence, the study recommends Dijkstra as the default shortest-path engine wherever route costs are guaranteed non-negative, with Bellman-Ford reserved for scenarios that require negative-edge support or negative-cycle validation of the underlying cost model.

13.2 Limitations

- All experiments were run on synthetically generated random graphs rather than a real-world road/logistics dataset; real transportation networks may exhibit different structural properties (e.g., approximate planarity, hub-and-spoke clustering) that could shift the exact speedup factors observed.
- Dense-graph experiments were capped at 800 vertices for practical runtime reasons; the largest dense graph tested still had over 125,000 edges, which is sufficient to establish the trend but does not probe extremely large-scale (millions of edges) behaviour.
- Timing measurements were taken on a single shared execution environment; while each configuration was repeated 3 times and averaged to reduce noise, absolute millisecond figures should be read as indicative rather than as a hardware-independent benchmark.
- The current Dijkstra implementation uses Python's built-in binary heap (heapq); a Fibonacci-heap implementation, which offers the theoretically better $O(E + V \log V)$ bound, was not implemented, since Python's standard library and typical practical use rarely justify a Fibonacci heap's higher constant-factor overhead.
- The negative-edge experiments used a fixed 5% negative-edge fraction; sensitivity to the proportion and magnitude of negative weights was not exhaustively swept.

13.3 Possible Improvements

- Validate the findings against a real, publicly available road-network or logistics dataset (e.g., an OpenStreetMap-derived regional road graph) to confirm the speedup factors generalise beyond synthetic random graphs.
- Implement and benchmark a Fibonacci-heap or pairing-heap variant of Dijkstra to empirically test whether the theoretically better asymptotic bound translates into a practical advantage at the tested graph scales.
- Extend the study to the all-pairs shortest-path setting (e.g., Floyd-Warshall or Johnson's algorithm, which itself internally combines Bellman-Ford for edge re-weighting with Dijkstra for the pairwise

queries) to reflect a logistics company's need to plan many source-destination pairs simultaneously.

- Add a parallel/multi-threaded or GPU-accelerated variant of Bellman-Ford's edge-relaxation pass, which parallelises naturally, to test whether it can close some of the performance gap with Dijkstra on very large dense graphs.
- Incorporate real-time, dynamically changing edge weights (e.g., live traffic conditions) and evaluate incremental / dynamic shortest-path algorithms as a further extension.

14. Individual Contribution of Group Members

(Replace the placeholder names and adjust rows to match your actual team size and how the work was really split. Percentages across a row's 'Approx. Contribution' column should reflect genuine relative effort, not an automatic even split, and should be agreed on by the whole team.)

Member Name / Roll No.	Primary Responsibilities	Approx. Contribution
[Member 1 Name]	Problem formulation; Dijkstra's algorithm implementation & instrumentation; correctness test cases 1-3	____%
[Member 2 Name]	Bellman-Ford implementation & negative-cycle detection; correctness test cases 4-6; negative-edge experiments	____%
[Member 3 Name]	Synthetic graph generator; experimental harness; results CSV, plots and figures	____%
[Member 4 Name]	Complexity analysis (Big-O/ Ω / Θ derivation); analysis, comparison and recommendation write-up; SDG mapping	____%
All members	Report compilation, review, and one-page individual reflections (Section 16)	Equal

15. References

15.1 Core Algorithm References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. -- Chapters on Single-Source Shortest Paths (Dijkstra's and Bellman-Ford algorithms), asymptotic notation.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271.
- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87-90.
- Ford, L. R. Jr. (1956). Network Flow Theory. Paper P-923, RAND Corporation.

15.2 Software, Tools and Datasets

- Python Software Foundation. Python 3.11 Language Reference and Standard Library (heapq, math, time, tracemalloc, dataclasses modules). <https://docs.python.org/3/>
- The pandas development team. pandas: Python Data Analysis Library. <https://pandas.pydata.org/>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://matplotlib.org/>
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference*. <https://networkx.org/>
- Gansner, E. R., & North, S. C. (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11), 1203-1233. (Graphviz) <https://graphviz.org/>
- Dataset used: synthetically generated weighted directed graphs (this study's own code/graph_generator.py), seeded for reproducibility; no external third-party dataset was required, consistent with the assignment's 'publicly available or synthetically generated' constraint.

16. One-Page Individual Reflection

(Each team member must submit their own copy of this section in their own words -- duplicate the template below once per member and personalise every paragraph. The version below is a fully worked example/template covering all required elements: design decisions, challenges faced, learning gained, SDG connection, and contribution to the mapped Course Outcomes.)

Reflection -- [Member Name, Roll No.]

Design and Development Decisions

One of the first decisions I was involved in was how to make the comparison between Dijkstra's algorithm and Bellman-Ford genuinely fair rather than superficial. We agreed to instrument both algorithms with an identical RunResult structure (capturing time, relaxations, comparisons and peak memory) so that neither implementation had an unfair measurement advantage, and to run both algorithms on exactly the same generated graph in every experiment rather than regenerating graphs independently for each algorithm. I also contributed to the decision to build a spanning arborescence rooted at the source vertex when generating random graphs, after we discovered during testing that a naive random spanning tree could leave the source vertex with no outgoing edges at all, which would have silently invalidated some of our experiments.

Challenges Faced

The most significant challenge I personally faced was constructing a genuinely correct demonstration that Dijkstra fails on negative edge weights (Test Case 5). My first attempt at a counter-example graph did not actually break our heap-based implementation, because the vertex in question had not yet been popped from the heap when the cheaper negative-weight path was discovered, so the algorithm still produced the correct answer 'by luck'. I had to reason carefully about exactly when a vertex becomes 'finalised' in a heap-based Dijkstra implementation, and construct a graph where the negative improvement is only discovered after the affected vertex has already been popped and its outgoing edges already processed. This taught me that a textbook claim ('Dijkstra doesn't work with negative weights') is not automatically demonstrated by any graph with a negative edge -- it requires understanding the precise internal mechanics of the specific implementation to construct a valid counter-example.

Learning Gained

Beyond the mechanics of the two algorithms, this assignment gave me a much more concrete appreciation of the gap between theoretical worst-case complexity and observed practical performance. Our results showed that Bellman-Ford's measured running-time growth was noticeably gentler than the raw $O(V \cdot E)$ bound would predict, because of the early-termination optimisation and the small effective diameter of our random test graphs. This was a valuable lesson that asymptotic upper bounds describe the worst case, not necessarily the typical case, and that empirical benchmarking is essential to understand how an algorithm actually behaves on realistic inputs.

SDG Connection

Working on this assignment made the connection between algorithm design and SDG 9 (Industry, Innovation and Infrastructure) very tangible for me. Seeing a 4-8x difference in computation time between two correct algorithms on the same route-planning task made it clear that algorithmic choice is itself a form of infrastructure investment -- a logistics company that chooses the right algorithm for its network

characteristics can serve more route queries with the same computing resources, directly supporting more efficient and resilient transport infrastructure.

Contribution to Mapped Course Outcomes

This assignment directly strengthened my ability to meet CO1 by requiring me to derive, not just quote, the Big-O, Ω and Θ bounds for both algorithms from their loop and heap-operation structure. It strengthened CO2 by having me implement and reason about Bellman-Ford as a brute-force relaxation technique and compare its exhaustiveness against a more structured approach. It strengthened CO3 by having me design and implement a graph-based, priority-queue-driven algorithm (Dijkstra) and critically analyse its trade-offs against a more general graph technique, reaching Bloom's Levels 4-6 by analysing the theoretical/empirical gap, evaluating which algorithm suits which network scenario, and creating a working, tested, and documented implementation of both.

(Word count guidance: this reflection should fit approximately one page per member at standard report formatting -- adjust length accordingly when personalising.)